Developing Case-Based Reasoning for Structural Design

Mary Lou Maher and Andrés Gómez de Silva Garza
University of Sydney

CASE-BASED SYSTEMS ENABLE USERS TO RETRIEVE PREVIOUSLY KNOWN DESIGNS FROM MEMORY AND ADAPT THEM TO FIT THE CURRENT DESIGN PROBLEM. THE FOUR CASE-BASED DESIGN SYSTEMS DESCRIBED HERE ILLUSTRATE HOW VARIOUS IMPLEMENTATIONS ACHIEVE DESIGN ASSISTANCE OR DESIGN AUTOMATION OBJECTIVES.

CASE-BASED REASONING IS A problem-solving technique that makes analogies between a problem and previously encountered situations (cases) relevant to solving the problem. Using CBR as a design process model involves the subtasks of recalling previously known designs from memory and adapting these design cases or subcases to fit the current design context.

The detailed development of this process model for a particular design domain proceeds in parallel with the development of the case representation, the case memory organization, and the necessary design knowledge. The selection of an information representation paradigm and the details of its use for a problem-solving domain depend on the intended use of the information, the project information available, and the nature of the domain.

CBR could be used to develop and implement a CBR system. Although that sounds circular, if CBR is a viable approach to problem solving, it can be applied to the development of the reasoning system itself. Toward that end, this article presents four "cases" of case-based building design systems that we've developed at the University of Sydney: CaseCAD, CADsyn, Win, and Demex. These systems exemplify alternative case memory contents and organizations and provide insight into different potential implementations of the recall and adaptation subprocesses.

Implementing case-based structural design

Developing a CBR system starts with the identification of cases that can provide the basis for solving design problems. We began our CBR projects by locating a source of information on structural design projects—a well-known engineering consulting firm that has worked on many medium- to high-rise buildings in Sydney. The firm gave us access to documented project information—primarily design drawings—and the engineers who worked on the projects. Initially, this seemed to be an ideal situation because the engineers were readily available and open to the idea of our developing a case base from their projects.

As it turned out, collecting information about the project designs was much more difficult than collecting the specifications to the building contractors. In the building industry, documentation primarily communicates construction information rather than the designers' intentions about how the design is to behave. Information about the geometric and material properties of the proposed building is available, but explicit functional or behavioral information is not. Therefore, we had to infer design information based on our theoretical understanding of how structural systems behave.

0885-9000/96/$4.00 © 1996 IEEE

We also found that before considering each project, we needed a template outlining the design description. That is, we had to decide ahead of time how to represent the case in memory and then try to match our expectations with the information actually available for a specific design project. Collecting case information is an iterative process that moves from collecting information to changing what we were looking for. So, although we thought that CBR system development starts with identifying cases, we discovered that first we had to determine what information we expected the cases to include and what the case-based reasoner's goal would be. At the same time, we had to allow for revisions of our expectations.

Some commonalities of our four systems reflect our basic approach to CBR. At the most basic level, each system uses attribute-value pairs to represent case memory. The attribute-value pairs are grouped into subsystems and form hierarchies, both in case memory organization and in indexing. The systems group the attribute-value pairs according to the functional, behavioral, and structural roles they play in design reasoning. Although the basic representation paradigm is simple, the organization of cases and case hierarchies builds a more complex representation.

CaseCAD. The CaseCAD project's objective was to propose and implement a design case representation that helps designers find a set of cases relevant to a specified design problem. CaseCAD is a case-browsing tool: it lets users navigate through its memory to look at information that appears relevant to their current interests. It can also retrieve cases autonomously. To pursue the system objectives, we focused on a multimedia case representation to make information readily understood by designers.[1] We also chose flexible memory-indexing techniques that let designers use many different paths to retrieve a case.[2]

Case representation. We collected CaseCAD's design case base for one year, in parallel with system development. We discarded much of the collected data because it was too detailed for solving new conceptual design problems. Finally, we reviewed and collected data from 14 design projects. We implemented four of these projects in the final system to demonstrate recall and adaptation. We did not implement the remaining 10 because they were not necessary to demonstrate these subtasks.

Graphical representations play a vital role in the communication of design information among designers and other building professionals. Graphical media provide a convenient way to handle the complexity of the design description. The human visual system can rapidly capture the wealth of information implicitly and explicitly represented in images, sketches, and drawings. But there are two problems with relying entirely on graphical case representations. First, a graphical representation does not make the design's function or behavior explicit—it addresses only what the design looks like. Second, computers do not reason about complex graphical representations easily. Some case-based design systems, such as Fabel,[3] that use graphical representations for the user interface perform reasoning by converting the images to alternative symbolic representations.

THE ATTRIBUTE CATEGORIZATION PROVIDES AN EFFECTIVE WAY OF REASONING ABOUT THE VARIOUS DESIGN VARIABLES, BECAUSE EACH CLASS OF VARIABLES HAS A DIFFERENT SEMANTICS ASSOCIATED WITH IT.

CaseCAD uses an object-oriented, multimedia case representation. Each object includes a reference to CAD drawings and images, groups of attribute-value pairs, and text-based case descriptions. Cases are decomposed into subcases for easy partial recall and design part reuse. The system uses the attribute-value data for case organization and recall, and the graphics and text descriptions help the designer understand the case information. The attribute-value data also helps designers by summarizing important information in discrete packages and explicitly interpreting the graphical representations.

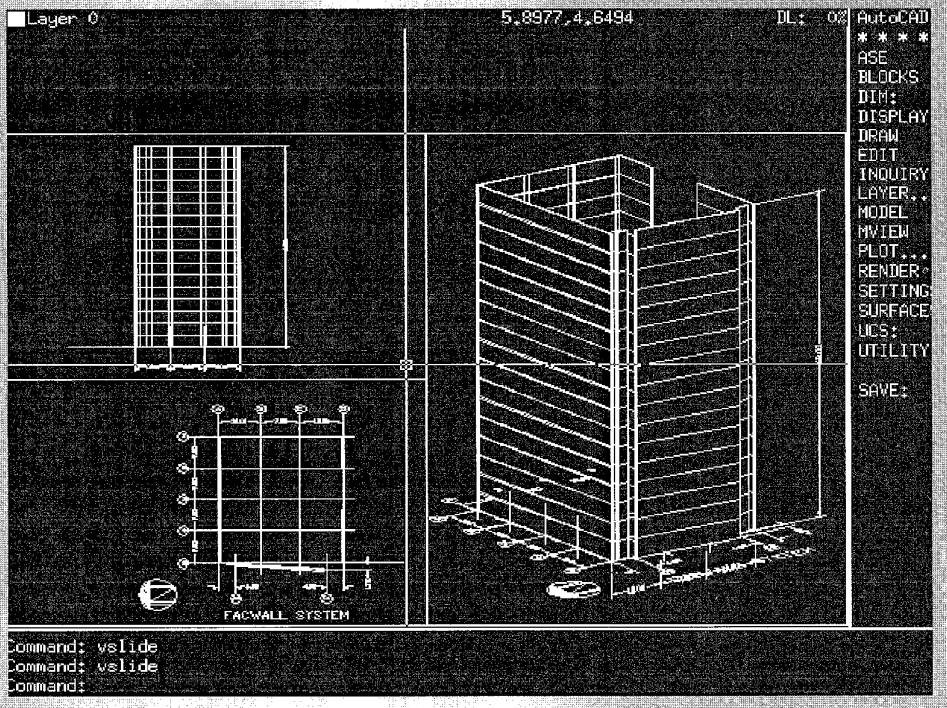
CaseCAD's object-based representation follows Gero's design prototype framework,[4] which classifies attributes by function, behavior, and structure. CaseCAD adds a fourth category, *relation attributes*, to facilitate implementation of hierarchies and other relational information. Function attributes represent the design's intended purpose—for instance, `support-building-type`, `grid-geometry`, and `resist-wind-load`. Behavior attributes represent the design's response to its environment (its performance); examples are `lateral-displacement`, `total-useable-area`, and `construction-cost`. Structure attributes are the design's physical features, such as `number-of-floors`, `width`, and `height`. Finally, relation attributes represent relationships among design cases, or between design cases and general concepts.

The attribute categorization provides an effective way of reasoning about the various design variables, because each class of variables has a different semantics associated with it. Making the distinctions explicit allows the system to deal with each type differently. Also, meaningful categorization of the vast number of variables that typically describe a building's structural design reduces the representation's complexity.

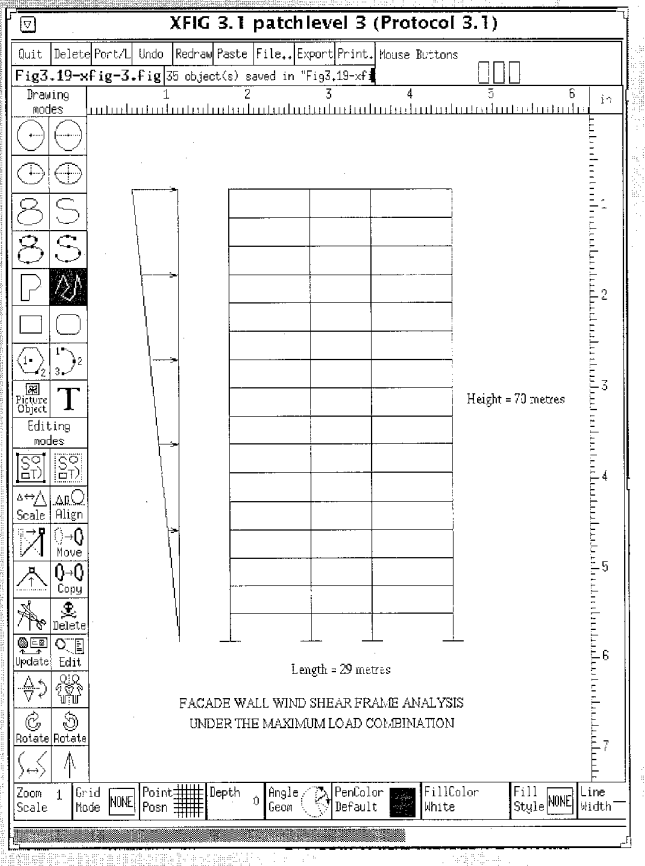
We implemented case representation in CaseCAD using Framekit, a frame-based representation language derived from Common Lisp. CaseCAD's graphical case representation uses AutoCAD, a general-purpose modeling tool for 3D design visualization, and XFIG, a 2D drawing tool. In CaseCAD, AutoCAD communicates a building design's structural aspects to the user, and XFIG illustrates behavioral information about a case. Figure 1a shows a CaseCAD AutoCAD drawing of a building's structure. Figure 1b shows an XFIG drawing of the same structure's behavior. Figure 1c shows an object-oriented case representation.

In addition to knowledge about specific designs in the form of cases, CaseCAD's memory contains general knowledge about classes of designs, in the form of models. For instance, one model represents CaseCAD's knowledge about medium-rise office buildings. CaseCAD models use object-based representation only, not graphical representation. The object-based model representation reflects the case representation—that is, it consists of sets of attributes in three categories. However, a model's attributes can take values from a finite list or range of values, rather than being associated with one specific value. For instance, the attribute `number-of-floors` has a value of 30 in the `Inn-on-the-Park` case representation, but the

Figure 1. Representations used by CaseCAD: (a) AutoCAD drawing of a building's structure; (b) XFIG drawing of a building's behavior; (c) excerpt of an object-oriented case representation.



(a)



(b)

```
SUBCASE1001
DESCRIPTION: The floor system is a com-
bination of standard Rescrete precast
Formplank panels supported by custom-
made precast beam shells and by facade
wall panels. The ends of the Formplank
panels rested on either corbels on the
facade wall units or the upturned edges
of the precast beam shells. These pre-
cast elements were used to produce a
one-way slab, one-way beam system with
structural detailing providing continu-
ity between beams, precast columns, and
facade panels.
RELATION
    part-of: 370Pitt
    instance-of: floor-system
FUNCTION
    transmit-dead-load: 3 kPa
    transmit-live-load: 3 kPa
BEHAVIOR
    fire-resistance-level: 2 hrs
    deflection: 18 mm
    shear-stress: 0.98 Mpa
    xfig-file: "370Pitt/floor-system.fig"
STRUCTURE
    material: precast-concrete
    start-level: 1
    end-level: 17
    span-type:one-way
    maximum-span: 11 m
    typical-span: 9 m
    floor-depth: 250 mm
    span-to-depth-ratio: 44
    maximum-aspect-ratio: 4.2
    acad-file: "370Pitt/floor-system.sld"
```

(c)

```
FLOOR-SYSTEM
RELATION
  part-of: building
  instances: subcase1001, ...
FUNCTION
  transmit-dead-load: 2.9-3.1 kPa
  transmit-live-load: 2.9-3.1 kPa
BEHAVIOR
  deflection: 17-19 mm
  fire-resistance-level: 2
STRUCTURE
  material: precast-concrete, reinforced-concrete
  span-type: one-way
```

Figure 2. Partial CaseCAD design model.

same attribute has a range of 14 to 30 in the medium-rise office building model representation. In the sample subcase in Figure 1c, the relation attribute `instance-of` indicates the model of the general type of structure the subcase belongs to—specifically, `floor-system`. Every case in CaseCAD is an instance of a model known by the system. Figure 2 shows a partial design model.

Browsing and retrieving. CaseCAD's browsing mode enables the user to see the graphical information about a case, using AutoCAD or XFIG, or the object-based representation of the same case. A graphical interface provides menus listing names of cases known to the system, allowing the user to move from one case to another. Figure 3 shows the browser displaying both graphical and text descriptions of the case.

In the retrieving mode, the user gives the system a design problem specification, which CaseCAD uses as an index to retrieve relevant cases from case memory. Design problem specifications, like the object-based case descriptions and models, consist of a set of attribute-value pairs representing the functional, behavioral, and structural requirements of new structural designs. The system measures a model's or a case's relevance to a given problem by finding matches between the attributes and values in the problem specification and those in the object-based description of the memory item.

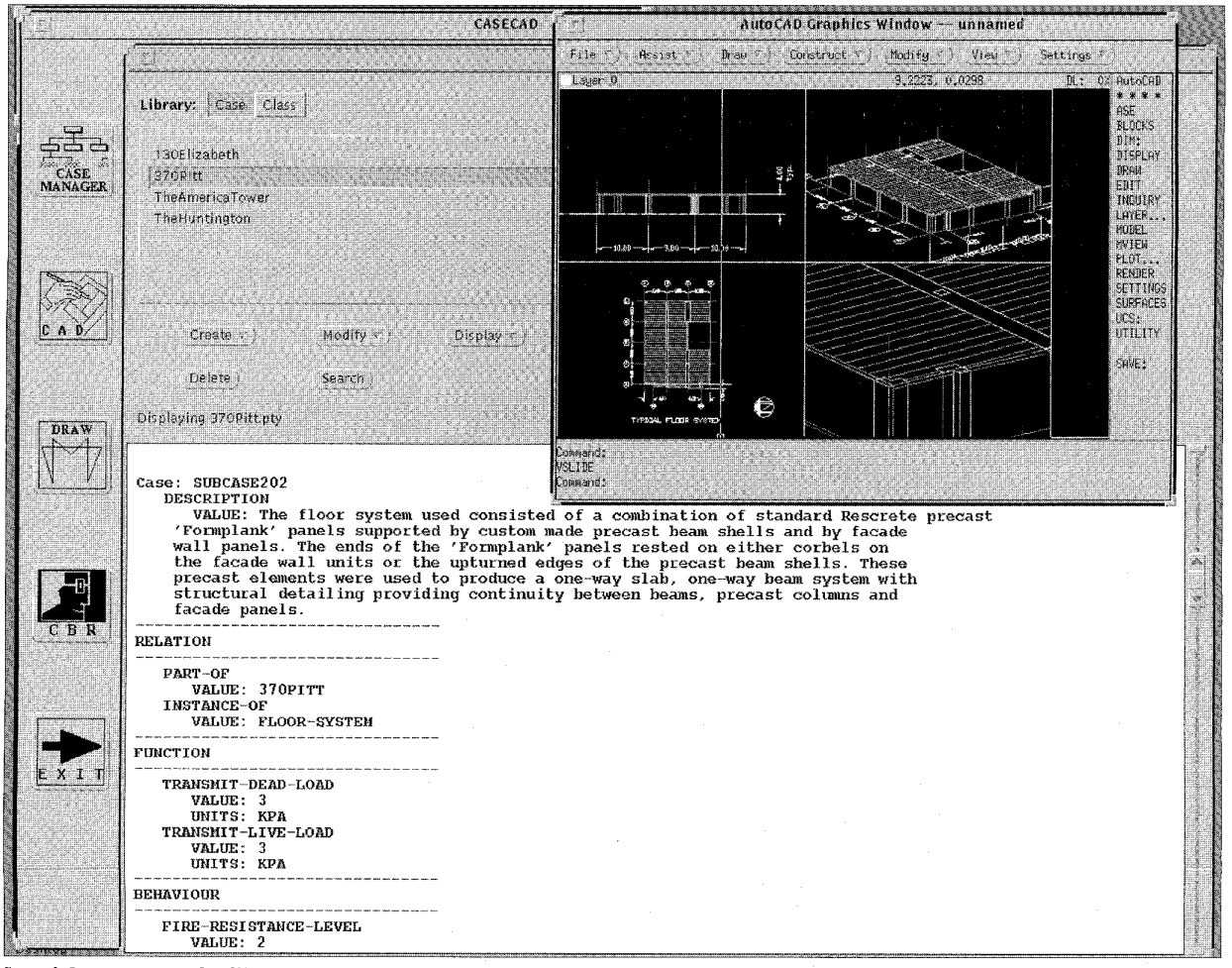


Figure 3. Browsing a case in CaseCAD.

Specifications:

```
FUNCTION
   SUPPORT-GRID-GEOMETRY: (= RECTANGULAR)
   MAXIMUM-SPAN: (= 9 M)
   SUPPORT-BUILDING-TYPE: (= OFFICE)
STRUCTURE
   MATERIAL: (= REINFORCED-CONCRETE)
BEHAVIOR
   COST-OF-CONSTRUCTION: (<= $20 MILLION)
```

Retrieving cases using the FUNCTION specifications:

requirement	no.-of-matchings
SUPPORT-BUILDING-TYPE = OFFICE	7
MAXIMUM-SPAN = 9 M	4
SUPPORT-GRID-GEOMETRY = RECTANGULAR	4

The cases retrieved based on FUNCTION specifications:

subcase	type	no.-of-matchings
130ELIZABETH	MEDIUM-RISE-OFFICE-BUILDING	3
SUBCASE102	RECTANGULAR-GRID-SYSTEM	1
370PITT	MEDIUM-RISE-OFFICE-BUILDING	3
SUBCASE201	RECTANGULAR-GRID-SYSTEM	1
THEAMERICATOWER	HIGH-RISE-OFFICE-BUILDING	3
SUBCASE301	RECTANGULAR-GRID-SYSTEM	1
SUBCASE401	RECTANGULAR-GRID-SYSTEM	1
THEHUNTINGTON	HIGH-RISE-RESIDENTIAL-BUILDING	2

Figure 4. Case retrieval in CaseCAD.

The case that matches the largest number of requirements from the problem specification might not be the best case to use for adaptation. The categorization of attributes into function, behavior, and structure provides a way to focus the retrieval. Rather than using explicit weights to assign importance to each specification, CaseCAD lets the user focus on a relevant set of attributes and ignore others while retrieving cases.

Figure 4 shows an example of a user's problem specification and the system output, detailing cases and subcases that partially match the specification, and the number of requirements they matched. Also shown is the type of structure the case or subcase exemplifies—that is, the model of which the case or subcase is an instance. The retrieval in this example uses only functional attributes as a focus for determining relevance.

Case adaptation. CaseCAD does not perform case adaptation automatically. The designer can modify retrieved cases and use them as new cases in case memory. For the adaptation, the designer uses editing tools to change the object representation, AutoCAD to modify the CAD drawing, and XFIG to modify the images. When the designer adds the case to memory, CaseCAD automatically indexes the object representation, using the attribute-value pairs defined by the designer.

CADsyn. The CADsyn project's objective was to develop a design system that adapts a recalled case automatically to fit a new context.[5] To that end, we developed a case memory representation complementing earlier research on design knowledge representation. The focus on case adaptation was necessary to understand the types and extent of knowledge that would make a CBR system more than a memory-based system. That is, CBR could provide a process model for generating new designs, not just recalling relevant ones.

Case representation. CADsyn's case representation is somewhat similar to CaseCAD's. CADsyn decomposes cases into subcases,

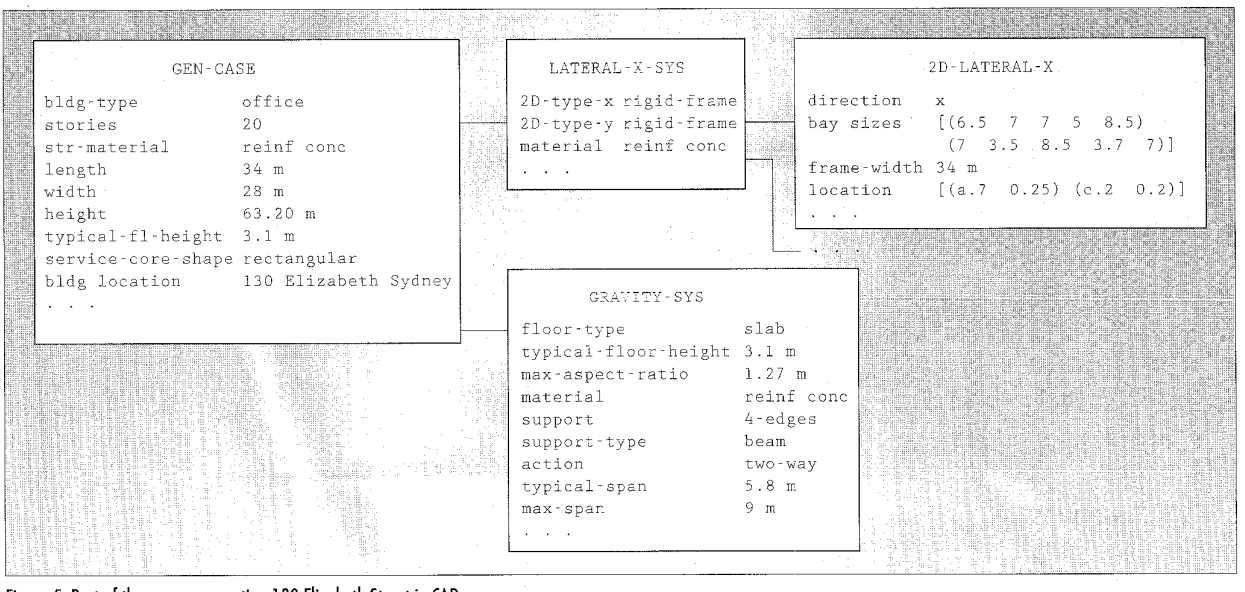


Figure 5. Part of the case representing 130 Elizabeth Street in CADsyn.

```
CONSTRAINT-1
   (bldg-type) in (office parking institution)
   (gravity-system/floor-type) in (flat-plate flat-slab)
CONSTRAINT-2
   (bldg-type) in (hotel apartment)
   (gravity-system/floor-type) = slab
CONSTRAINT-3
   (gravity-system/floor-type) in (flat-plate flat-slab)
   (gravity-system/action) = one-way
   (gravity-system/typical-span) < 10 m
CONSTRAINT-4
   (gravity-system/floor-type) in (flat-plate flat-slab slab)
   (gravity-system/typical-span) > 9 m
   (gravity-system/action) = two-way
CONSTRAINT-5
   (gravity-system/support) in (4-edges cols-only)
   (gravity-system/max-aspect-ratio) > 1.5
   (gravity-system/action) = two-way
```

Figure 6. CADsyn's constraint representation.

and the primary object-based representation uses attribute-value pairs to describe a case. There are major differences, however. First, because CADsyn focuses on case adaptation rather than browsing and retrieval, it does not use graphical, textual, or other human-amenable representations. Adapting the attribute-value pair representation is possible with knowledge-based techniques, but adapting graphical representations or text descriptions is difficult. Second, CADsyn cases are related to models, but we refer to the models as systems and subsystems. This is due to CADsyn's association with EDEsyn,[6] a representation of design synthesis knowledge using decomposition hierarchies called systems. Finally, CADsyn augments the case and system knowledge representation with constraints to adapt cases using a constraint satisfaction approach.

Most cases in CADsyn's memory are the same as in CaseCAD's. CADsyn's memory contains four cases, each with 10 to 14 subcases. The differences between CaseCAD's and CADsyn's case representations lie in the attribute-value pairs and the decomposition into subcases. In CADsyn, we chose attributes and their values according to their utility in implementing the case adaptation subtask and their involvement in constraints. Figure 5 shows an example of a case and a subset of its subcases.

Case retrieval. Design problem specification in CADsyn is the same as in CaseCAD: a set of requirements, in the form of attribute-value pairs, that a design should satisfy to be a valid solution to the problem. The system matches the problem specification with the case descriptions in its memory and retrieves cases and subcases with a partial match. It ranks the retrieved cases according to the number of matching features, favoring the closest match.

CADsyn uses two case-indexing structures: a requirements hierarchy and a subsystem list. The requirements hierarchy uses a conceptual clustering approach.[7] The system traverses the hierarchy according to the problem specification until it finds a list of matching cases. In other words, CADsyn does not search the case memory to match and retrieve cases, but rather searches a separate representation of the requirements hierarchy, which points to relevant cases. The subsystem list associates a subsystem label with a set of subcases that use that label. If CADsyn does not find a relevant case in the requirements hierarchy, it sends the problem specifications to the decomposition reasoner, which identifies a relevant set of subsystems. CADsyn then uses these subsystems to retrieve associated subcases and performs adaptation on the subcases rather than on an entire building design.

Design case adaptation. CADsyn uses constraint satisfaction to adapt design cases, representing constraints as a set of attribute-value pairs that identify an invalid design. If the constraints represented valid assignments of values to variables, they would only help verify the correctness of proposed designs. Using them to represent invalid value assignments helps detect exactly where problems lie in a proposed design, thus focusing attention on design aspects that need alteration to achieve a goal.

Figure 6 shows examples of constraints in CADsyn. `CONSTRAINT-2` can be read as "If the design uses a `floor-type` = `slab` and `bldg-type` = `hotel`, then the design is invalid." The constraints are represented as logical relationships among attribute-value pairs, rather than as numerical relationships.

CADsyn's constraint satisfaction modifies a proposed complete design description, based on a retrieved case or subcase, to satisfy the design specifications. Modifying the retrieved case or subcase to match the design specifications typically introduces constraint violations that must be resolved. For example, if a problem specification for a hotel retrieves an office building case, the system must change the value of the attribute `bldg-type` of the retrieved case to `hotel` as the first step in adapting the case to the user's requirements. Additionally, let's assume that the `floor-type` of the retrieved case has a value of `slab` and that the problem specification did not include any reference to the floor type the hotel should have. The value of the attribute `floor-type` in the partially constructed solution would now violate `CONSTRAINT-2`.

Thus, CADsyn's design case adaptation takes an initially inconsistent assignment of values for design attributes and repairs constraint violations through a search process until it achieves a consistent assignment. Reflecting the proposed design's hierarchical representation as a composition of subcases, the constraint satisfaction adaptation generates a feasible design by modifying invalid subsystems (subcases) in the inconsistent potential design. CADsyn detects invalid subsystems by testing constraints and modifies them using its general knowledge of the valid values of design attributes. Because the constraints are not numerical, the system must validate subsystems incrementally, and determining which invalid subsystem to modify first is a central task.

Figure 7 illustrates the subtasks constituting CADsyn's computational model of design case adaptation as constraint satisfaction. These subtasks iterate until they reach a consistent solution to the user's design problem. The figure also shows the knowledge supporting each task. The constraints provide the knowledge necessary to check constraints, find invalid systems, and select a feasible combination of attribute values for a subsystem design. The feasible combination is the combination that violates the fewest constraints. The system uses decomposition knowledge to find invalid systems, to find valid combinations for new design descriptions, and to propagate the effects of the new subsystem design. It uses heuristic rules to select a subsystem to modify. Heuristic rules are necessary because

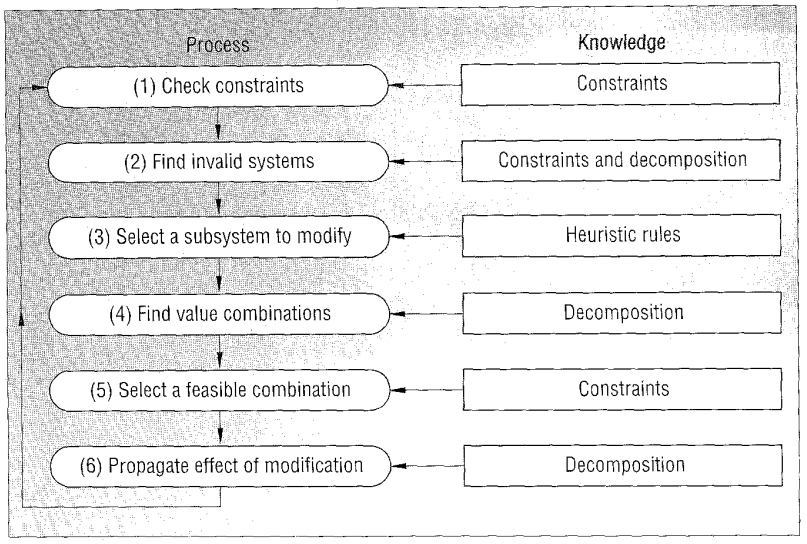


Figure 7. CADsyn's constraint satisfaction process.

the constraints are themselves heuristics, making the design space difficult to analyze.

The complexity of CADsyn's constraint satisfaction process accommodates adaptation of multiple subcases in a hierarchical design representation in which constraints represent logical relationships among design variables. CADsyn's use of constraints contrasts with other systems. For example, Julia, a case-based reasoner for meal planning, uses constraints to adapt cases, representing the meal as a whole case.[8] Cadre, a case-based reasoner for floor plan design, uses constraints to represent numerical relationships among dimension variables.[9]

Win. The Win project's objective was to develop a case-based design assistant with a structural engineering perspective.[10] The project focused on case memory content for a specific structural design aspect: design and layout of wind-resisting systems in medium- and high-rise buildings. The result is a case memory representation specific to that domain.

Case representation. Three of the structural design cases in CaseCAD are represented in the Win system, where each case has approximately four subcases. In Win, cases have more specific information about the engineering performance of lateral load systems than in CaseCAD. Win's case base contains more numerical information about stresses and loads and less high-level symbolic information about the architectural layout. Like CaseCAD, Win's memory contains knowledge in the form of both models and cases, described as attributes and values. Attributes fall into four categories: relation, function, behavior, and structure. Both models and cases are decomposed into subsystems.

Models play different roles in CaseCAD and Win. CaseCAD models are decomposed into subsystems that facilitate the acquisition and retrieval of cases. Win models assist case memory organization and problem specification. Specifically, Win models are defined according to two factors that greatly influence the design of a structure's wind-resisting system: the building's dimensions and the position of the building's core system with respect to the rest of its structure. Thus, Win has four models: medium- and high-rise buildings and central- and offset-core structures. These models' primary purpose is to partition case memory according to the intended use.

Figure 8a shows a model corresponding to the class of medium-rise structures with an offset core. Figure 8b shows a Win case representation of the Inn on the Park, a 30-story hotel and office building in Sydney. Figure 8c shows how Win uses models to partition case memory.

Indexing and retrieving. Case retrieval is an interactive, iterative process in Win. First, Win prompts the user for a set of problem requirements. For each of several important attributes (`number-of-floors`, `width`, `length`, `floor-to-floor-height`, and so on), Win asks the user to input its value, if known. The resulting set of attribute-value pairs becomes the problem specification, which Win uses as an index to its memory. Win first retrieves models or submodels that match the problem requirements before retrieving cases or subcases that are instances of the models. After model retrieval, Win considers the difference between the user's problem specifications and the extent and range of values of attributes in the models. Win requests additional information, or clarification, before initiating case retrieval.

This model-based case indexing allows the system to ignore cases and subcases that are completely irrelevant to a problem and efficiently find those that are relevant. Both the model memory and the case memory are flat structures—that is, retrieval proceeds by attempting to match the index with each item in memory successively. The model memory partitions the case memory. The small number of models and cases in Win's memory makes this flat memory organization feasible; more models would make it inefficient. The user always controls the use of Win's model and case bases and the retrieval and selection of cases.

Demex. The Demex (Design by Memory Exploration) project's objective was to consider flexible retrieval and memory exploration.[11] In this project, we developed the strategies of *index elaboration* and *index revision*, which we'll describe in more detail later. With the Demex case memory, a designer can explore and develop an improved design problem specification, rather than develop a design solution. The system reflects real-world design problem solving, in which a designer does not usually start with a well-specified problem and then move toward a solution, but rather the problem requirements evolve as the designer explores alternative solutions.[12]

Case memory representation. Demex uses CaseCAD's design cases and models, partitioning cases and models into subsystems and categorizing attributes as functional, behavioral, structural, and relational. Because of Demex's highly interactive nature, we devised a memory organization that allows efficient and flexible access to cases and subcases.

In Demex, three memories hold conceptually different types of information: an associative memory that relates the function, behavior, and structure categories with design attributes; a model memory; and a case memory. Although the three memories are stored as separate entities in their own data struc-

tures to increase efficiency and avoid redundancy, they are interrelated. To begin with, we designed all three using the same attribute-value ontology. Additionally, each model in the model memory contains a pointer to all cases that are instances of that model, and each case in the case memory contains a pointer to the model of which that case is an instance. Cross-indexing of the semantically different knowledge types allows flexible and efficient traversal among them.

Demex's indexing scheme for model and case memory is a tree of attributes and values (see Figure 9). The system can rapidly retrieve relevant models and cases using the corresponding indexing tree to perform a two-step comparison for each attribute-value pair in the problem specification. The first step traverses the tree along a given attribute arc; the second step traverses it along an arc corresponding to a value for that attribute, retrieving a set of cases with the specified attribute-value pair. If only some categories of attributes are of interest, the case retrieval ignores requirements whose attributes belong to an uninteresting category. We designed this three-memory organization and indexing-tree structure especially to support the retrieval strategies we'll describe next.

Index elaboration and revision. Restructuring, reconsidering, and reformulating a problem's specifications are an important aspect of design problem solving. Demex guides the user in reformulating structural design requirements by improving the user's understanding of the problem.

Demex uses two problem reformulation strategies to provide a better understanding of a design problem: model-based index elaboration and case-based index revision. These strategies evolved from discoveries we made in the CaseCAD project about the need for flexible case retrieval.[2]

Making an analogy between the current problem specification and general knowledge in memory, model-based index elaboration retrieves one or more models from memory. The models can provide a better description of the design problem in two ways. First, items retrieved from memory usually contain more information (for example, many more design variables to describe a structure) than does the initial problem specification. The designer can expand the set of specifications with this information.

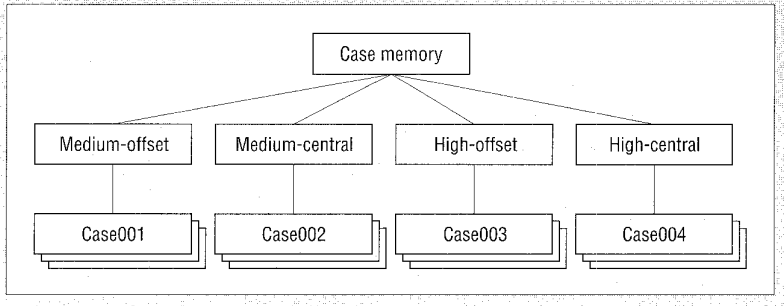
Second, models provide ranges of acceptable values associated with the design

```
***MEDIUM-OFFSET-BUILDING***
FUNCTION
   SUPPORT-BUILDING-TYPE: office, hotel, residential, other
BEHAVIOR
   ABOVE-GROUND-LEVELS: 14-30
   ABOVE-GROUND-HEIGHT: 40-120 m
   CRITICAL-HEIGHT-RATIO: 1-8
   CORSYS-LOCATION-AT-CRITICAL-LEVEL: offset
   CORSYS-AREA-RATIO-AT-CRITICAL-LEVEL: 0.01-0.5
   CORSYS-OFFSET-RATIO-IN-CRITICAL-DIRECTION: 0.1-1
   CORSYS-LENGTH-RATIO-AT-CRITICAL-LEVEL: 0.1-10
   CRITICAL-WIND-DIRECTION: ew, ns
   CRITICAL-WIND-SPEED: 30-60 m/s
STRUCTURE
   OVERALL-LENGTH: 15-120 m
   OVERALL-WIDTH: 15-120 m
   CORSYS-OVERALL-LENGTH: 2.5-25 m
   CORSYS-OVERALL-WIDTH: 2.5-25 m
   CRITICAL-WIND-AREA: 600-14400 m2
```

(a)

```
Case: INN-ON-THE-PARK
DESCRIPTION: The Inn on the Park is a 30-level interna-
tional hotel and commercial office building in the central
business district of Sydney. This development provides 14
levels of accommodation with 250 4-story rooms, occupying
11,700 square meters of floor area; 5 levels of commercial
office space comprising approximately 5,000 square meters
of floor area; 3 levels for hotel facilities; and 7 base-
ment levels for 170 cars. The wind-resisting structure sys-
tem is composed of a core system and a shear wall system.
RELATION
   INSTANCE-OF: MEDIUM-OFFSET-BUILDING
FUNCTION
   SUPPORT-BUILDING-TYPE: hotel
   HOTEL-SPACE (M2): 11700
   HOTEL-ROOMS: 250
   OFFICE-SPACE (M2): 5000
   UNDERGROUND-PARKING-CARS: 170
BEHAVIOR
   STRUCTURE-DESIGN-CODE: AS1480
   WIND-CODE: AS1170
   TOTAL-LEVELS: 30
   TRANSFER-LEVEL: level-8
   ABOVE-GROUND-LEVELS: 23
   UNDER-GROUND-LEVELS: 7
   OVERALL-SHAPE: quadrangular
   TOWER-SHAPE: quadrangular
STRUCTURE
   MATERIAL: insitu-rc
   OVERALL-LENGTH (M): 50
   OVERALL-WIDTH (M): 25
   STD-AREA (M2): 900
   ABOVE-GROUND-HEIGHT (M): 82
   UNDER-GROUND-HEIGHT (M): 20
```

(b)



(c)

Figure 8. Models and cases in Win: (a) design model representation; (b) case representation; (c) using models to partition case memory.

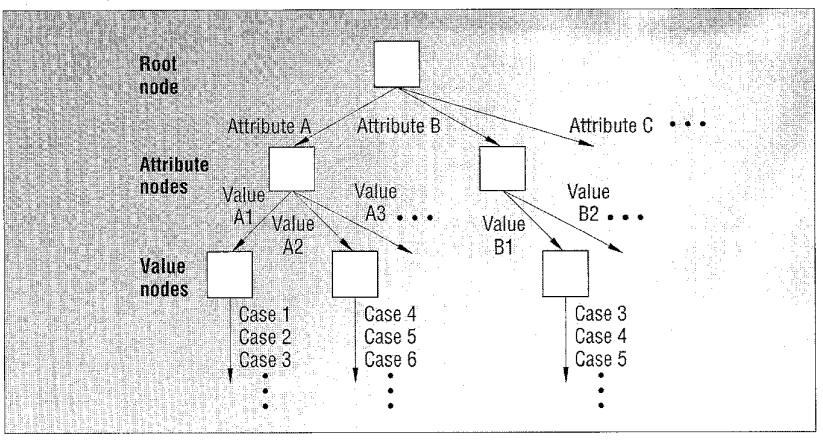


Figure 9. Demex case-indexing tree.

The example in Figure 11, which considers the design of a building's wind-resisting system, illustrates the case-based index revision strategy for problem reformulation. Demex retrieves the case shown in Figure 11b. To achieve wind resistance in structures, the retrieved case uses vertical trusses, not the rigid frames of the original specifications. So, the designer can modify the specifications (see Figure 11c).

In version 2, the attributes `number-` variables that describe the models. The user compares the values in the problem specification and the retrieved model's range of values. Attributes for which the specification value is not in the acceptable range are critical attributes. The user can prune the noncritical parts of a problem specification to focus on the important parts, resulting in a smaller design space.

Figure 10 illustrates model-based index elaboration. The user, interested in designing a medium-height office building, describes the problem to the system (see Figure 10a). Demex uses these problem specifications to retrieve a relevant model. For this example, the best match in the model memory is `medium-rise-office-buildings`, parts of which are shown in Figure 10b.

The retrieved model has more attributes than the user gave in the problem specifications—for example, `location-of-core`, `material`, and `height-to-width-ratio`. If the values are known for any of these attributes, adding them to the problem requirements can improve the focus of case retrieval. Let's assume that Figure 10c shows the user's new problem specifications, with relevant attributes (and their values) added.

A comparison of the problem specifications, version 2, with the design model shows that a value of 2.5 m for the `floor-to-floor-height` attribute is outside the valid range. This identifies a critical attribute that becomes the new problem specifications (see Figure 10d).

The case-based index revision strategy uses cases to modify the problem specifications. Modifications include adding, removing, and altering some or all of the attribute-value pairs in the problem specifications. Altering an attribute-value association could involve modifying the attribute's name, value, or both.

```
Function
Support-building-type:        office
Support-grid-geometry:        rectangular
Structure
Number-of-stories:            30
Overall-width:                23 m
```

(a)

```
Model: medium-rise-office-buildings
Function
Support-building-type:      office
Resist-wind-load:           (1.3 ... 1.7 kPa)
Support-grid-geometry:      (rectangular, circular, ...)
Behavior
Net-area-of-usable-space:   (8000 ... 25000 m2)
Structure
Number-of-stories:          (15 ... 30)
Height-to-width-ratio:      (2 ... 13)
Overall-width:              (20 ... 100 m)
Floor-to-floor-height:      (3 ... 4.5 m)
Material:                   (steel, reinforced-concrete, ...)
Floor-system-type:          (flat-plate, flat-slab, ...)
Wind-system-type:           (core, rigid-frame, ...)
Location-of-core:           (exterior, central, ...)
```

(b)

```
Function
Support-building-type:        office
Support-grid-geometry:        rectangular
Structure
Number-of-stories:            30
Overall-width:                23 m
Floor-to-floor-height:        2.5 m
Location-of-core:             central
```

(c)

```
Structure
Floor-to-floor-height:        2.5 m
```

(d)

Figure 10. Model-based index elaboration in Demex: (a) version 1 of problem specifications; (b) parts of best match retrieved from model memory; (c) version 2 of problem specifications; (d) version 3 of problem specifications.

of-bays and bay-width in the specification of the rigid frame become number-of-panels and panel-width. In one step, some parts of the description have changed, either in their values or their attributes, while others have remained intact. In this example, no parts of the original specification were dropped (although an alternative way of viewing the altering of an attribute-value association is that it was removed and replaced by an analogous association at the same time).

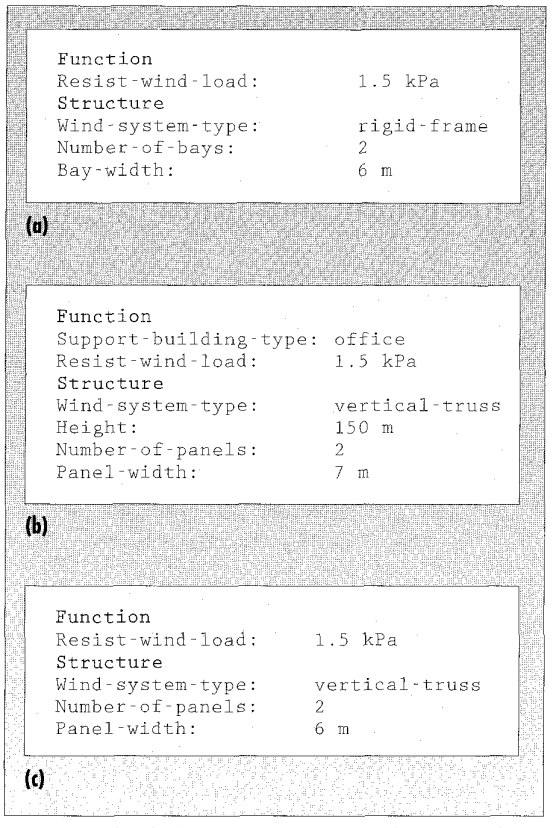


Figure 11. Case-based index revision in Demex: (a) initial problem specifications; (b) retrieved case; (c) second problem specifications.

System options

Our four projects and the experiences we gained from them can themselves serve as case studies on the development of case-based design tools. Although we cannot say that one approach is better than another, their differences point to options in developing CBR design systems.

Representation in case-based design. On the surface, all four systems described here use a similar representation paradigm; however, they have important differences. The similarities are the use of attribute-value pairs for representing a case or subcase and the use of models to support CBR. The attribute-value pair description allows indexing and adaptation of specific cases by means of relatively simple pattern matching based on string comparisons of attributes and values. In addition to the attribute-value pairs, the systems (except CADsyn) document cases with drawings, images, and text descriptions. One way of viewing the attribute-value pairs is to consider them the index to the more comprehensive case description.

The systems do not describe cases in terms of lessons learned but in a formalism relevant to design, such as intended function, actual behavior, and structural attributes. The selection of attributes in a case depends on the other knowledge necessary to support CBR. The attributes in a case description must match those in the model descriptions for the system to recognize that the knowledge in the models is relevant to the case. In CADsyn, the attributes in the case representations also must be the same as those included in the constraints used to identify constraint violations.

The differences among the systems include their case memory organization and the way they use indexes for case retrieval. In CaseCAD and CADsyn, memory is organized as lists of cases and models. All attribute-value pairs are valid indexes, and case retrieval consists of matching all or part of the problem specifications with each case and subcase. This approach is appropriate when case memory contains a limited number of cases and inappropriate when case memory becomes very large.

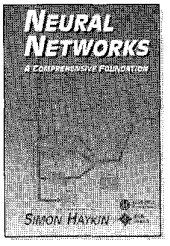
In Win, the models partition case memory so that the retrieval process first matches the problem specifications with each model. Assuming there are fewer models than cases, this scheme is more efficient than comparing each case. This approach allows only a comparison of the attribute-value pairs in the models; it is valid when the models adequately describe the functional decomposition of relevant cases and there are few models. The use of models to partition case memory limits retrieval to cases that are similar; it does not allow dissimilar but interesting cases to be retrieved.

Demex's tree structure, with attributes and values as branches, is the most efficient representation of a large case memory, because the number of cases does not affect the time needed to find partial matches.

Each system uses models differently. In CaseCAD, the models provide the template for identifying case information during the acquisition of new cases. CADsyn uses models to perform constraint satisfaction. Each model provides the domain space for the attributes in the case description. These attribute domains are the alternative values for fixing violated constraints. In Win, the models partition case memory and provide a heuristic grouping of indexes for problem elaboration and case retrieval. In Demex, the models provide the space from which index elaboration either refines and adds detail to the problem description or identifies critical requirements. Although the specific roles of models differ in each system, the general role of models is to support the reasoning aspects of case-based design.

Reasoning in case-based design. The purpose of reasoning in case-based design can range from design assistance to design automation. CaseCAD, Win, and Demex focus on design assistance through a memory-based system for the recall of similar designs. The reasoning in these systems relies on the user to guide retrieval, modification of problem specifications, and selection of a case to be pursued further. Recording problem specifications and comparing them to cases and models in memory, the reasoner's main task is memory retrieval.

CADsyn's reasoning aims at modeling design synthesis, which is closer to the design automation end of the spectrum. CADsyn's constraint satisfaction approach directly contrasts with CaseCAD's adaptation approach, in which the user changes the case, unsupported by the case-based reasoner. CADsyn is not a fully automated design system, because the user must select the appropriate case or subcase to be adapted and can override the system's selection of the



Neural Networks: A Comprehensive Foundation

by Simon Haykin

Represents the most comprehensive treatment available of neural networks from an engineering perspective. This thorough, well-organized, and completely up-to-date book examines all the important aspects of this emerging technology. The text features comprehensive chapter introductions, numerous computer experiments, network problems and solutions, photographs and illustrations and a bibliography and a thorough glossary that will reinforce the key concepts presented in this book.

Contents: Introduction • Learning Process • Correlation Matrix Memory • The Perceptron • Least-Mean-Square Algorithm • Multilayer Perceptrons • Radial Basis Function Networks • Recurrent Networks Rooted in Statistical Physics • Self-Organizing Systems: Hebbian, Competitive Learning, and Maximum Information Preservation • Modular Networks • Temporal Processing • Neurodynamics • VLSI Implementations of Neural Networks • Appendixes • Bibliography and References • Index

720 pages. 1994. Hardcover.
ISBN 0-02-352761-7.
Catalog # BP05687
$55.95 Members / $69.95 List

IEEE COMPUTER SOCIETY

Call toll-free:
1-800-CS-BOOKS
Fax: +1-714-821-4641

invalid system to be modified. CADsyn's reasoning process for design case adaptation, when made explicit, provides alternative design solutions for the designer.

THE FOUR SYSTEMS DESCRIBED here do not represent an exhaustive list of options for implementing case-based design. Although we used similar methods to represent design information and the same design cases, a variety of issues led us to develop different systems. We concentrated on structural design representation so that we could address problem formulation and solution generation. Using CBR for problem formulation led us to consider the role of cases in defining the scope of the problem. Using CBR for generating solutions led us to consider what changes and what stays the same in case adaptation, as well as the knowledge needed to make these decisions and methods of automating the process. The practicality of CBR systems for use by professionals depends on further developments in technology, such as multimedia database management support, and further developments in design knowledge representation.

References

1. M.L. Maher and B. Balachandran, "Multimedia Approach to Case-Based Structural Design," *J. Computing in Civil Engineering*, Vol. 8, No. 3, July 1994, pp. 359–376.
2. M.L. Maher and B. Balachandran, "Flexible Retrieval Strategies for Case-Based Design," in *Artificial Intelligence in Design '94*, J.S. Gero and F. Sudweeks, eds., Kluwer Academic, Dordrecht, The Netherlands, 1994, pp. 163–180.
3. A. Vob et al., "Retrieval of Similar Layouts," in *Artificial Intelligence in Design '94*, J.S. Gero and F. Sudweeks, eds., Kluwer Academic, 1994, pp. 625–640.
4. J.S. Gero, "Design Prototypes: A Knowledge Representation Schema for Design," *AI Magazine*, Vol. 11, No. 4, Winter 1990, pp. 26–36.
5. M.L. Maher and D.M. Zhang, "CADsyn: A Case-Based Design Process Model," *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, Vol. 7, No. 2, 1993, pp. 97–110.
6. M.L. Maher, "Engineering Design Synthesis: A Domain Independent Representation," *Artificial Intelligence in Engineering Analysis and Manufacturing*, Vol. 1, No. 3, 1987, pp. 207–213.
7. R.S. Michalski, "Learning from Observation: Conceptual Clustering," in *Machine Learning: An Artificial Intelligence Approach*, R.S. Michalski, J.G. Carbonell, and T.M. Mitchell, eds., Morgan Kaufmann, San Francisco, 1983, pp. 331–363.
8. T.R. Hinrichs and J.L. Kolodner, "The Role of Adaptation in Case-Based Design," *Proc. DARPA Workshop on Case-Based Reasoning*, Morgan Kaufmann, 1991, pp. 121–132.
9. B. Faltings et al., "Case-Based Representation of Architectural Design Knowledge," *Proc. DARPA Workshop on Case-Based Reasoning*, Morgan Kaufmann, 1991, pp. 307–316.
10. S.H. Wang, *Win: A Case-Based Design Approach to the Structural Design of the Wind Systems of Buildings*, master's thesis, Univ. of Sydney, Australia, 1995.
11. A. Gómez de Silva Garza and M.L. Maher, "Design by Interactive Exploration Using Memory-Based Techniques," to be published in *Knowledge-Based Systems*, Vol. 9, No. 1, 1996.
12. V. Goel and P. Pirolli, "The Structure of Design Problem Spaces," *Cognitive Science*, Vol. 16, 1992, pp. 395–429.

Mary Lou Maher is codirector of the Key Centre of Design Computing and an associate professor in the Department of Architectural and Design Science at the University of Sydney, Australia. Her research interests include case-based reasoning in design, modeling creative design using genetic algorithms, and computer support for collaborative design. She is the editor or coauthor of eight books and over 100 papers on design computing. She received her BS in civil engineering at Columbia University and her MS and PhD in Civil Engineering at Carnegie Mellon University. She can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; phone 61 (2) 351-4108; fax 61 (2) 351-3031; mary@arch su edu au; http://www.arch.su.edu.au/~mary/.

Andrés Gómez de Silva Garza is a PhD student in the Department of Architectural and Design Science at the University of Sydney. His research interests include case-based reasoning, genetic algorithms, and design science. He received his BSc in computer engineering from the National Autonomous University of Mexico in 1991, and his MSc in computer science from the Georgia Institute of Technology in 1993. He is a member of the AAAI. He can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; phone 61 (2) 351-2053; fax 61 (2) 351-3031; andres@arch.su.edu.au.